

EV Charger Simulator

Validate OCPP charging operations without physical hardware

What it is

The EV Charger Simulator is a production-focused **OCPP 1.6J** simulation toolkit that behaves like real charging stations over WebSocket. Teams use it to test onboarding, charging sessions, billing logic, and remote command handling before deploying to live infrastructure.

The problem it solves

Testing EV charging software with physical hardware is expensive, slow, and hard to scale:

- ⌘ A hardware test lab costs **<50,000 - <200,000** (<500 - <5,000 per charger).

- ⌘ Real-world integration tests take **6 - 12 weeks** to set up with physical equipment.

- ⌘ Edge cases - concurrent sessions, network drops, firmware resets, billing failures - are nearly **impossible to script repeatably** with hardware.

- ⌘ A single billing defect across 100 charging stations can generate **hundreds of incorrect invoices per day** before it is caught.

Why teams use it

- ⌘ **Launch faster:** test charger workflows on day one, without waiting for devices in the field.

- ⌘ **Lower QA cost:** run repeatable automated scenarios instead of manual hardware test cycles - **zero hardware, zero unit cost, 5-minute setup**.

- ⌘ **Catch edge cases early:** validate resets, reconnects, meter bursts, and fleet surges in controlled environments.

- ⌘ **Scale confidently:** simulate dozens of connectors and **hundreds of stations in parallel on a single machine**.

Core capabilities

- ⌘ Full charger lifecycle support:

- 'BootNotification', 'StatusNotification', 'Heartbeat'
- 'Authorize', 'StartTransaction', 'MeterValues', 'StopTransaction'

- ⌘ Handles server-initiated commands, including:

- 'RemoteStartTransaction', 'RemoteStopTransaction', 'Reset'
- 'ChangeConfiguration', 'GetConfiguration'
- 'SetChargingProfile', 'ClearChargingProfile', 'ClearCache', 'UnlockConnector', 'GetDiagnostics', 'UpdateFirmware'

- ⌘ Meter values for realistic downstream testing:

- Energy (Wh), power (W), voltage (V), and current (A) streams - exactly what billing pipelines expect

Four usage modes

Mode What it does Best for

'boot' Keeps a simulated charger online, responding to every dashboard command in real time Manual QA, remote command testing

'session' Executes one complete charge session flow: Authorize Start MeterValues Stop End-to-end billing validation

'multi' Runs concurrent sessions across all connectors Smart charging, load balancing, per-connector billing

'fleet' Launches many chargers simultaneously Load testing, stress testing, CI/CD regression suites

Built for engineering and operations

â€¢ Supports local development, CI pipelines, and staging validation.

â€¢ **Accelerated time mode:** 1 simulated minute = 1 real second validate a **60-minute charging session in 60 seconds**, or run a full overnight regression suite that would take weeks with hardware.

â€¢ Helps verify platform behavior before touching customer-facing systems.

Simulator vs. physical hardware

Physical hardware EV Charger Simulator

Unit cost <500 <5,000 **<0**

Test lab setup Days to weeks **Under 5 minutes**

Concurrent chargers Limited by budget **Hundreds on a single machine**

Session duration Real time (60 min = 60 min) **Compressed (60 min = 60 sec)**

Edge case reproduction Manual, unreliable **Fully scripted, 100% repeatable**

CI/CD integration Not possible **Native GitHub Actions, GitLab, Jenkins**

Typical outcomes

â€¢ Faster integration with OCPP-compliant backends

â€¢ Better reliability of remote operations

â€¢ More predictable releases with fewer field regressions

â€¢ Improved confidence in billing and transaction analytics

Quick start

npm install

cp config.example.json config.json # add your tenant ID + registration token

`npm run dev boot # your virtual charger is live`

Run a full session with accelerated time:

`EV_SIM_ID_TAG=AABBCCDD EV_SIM_ACCELERATED=true npm run dev session`

Stress-test with a 50-charger fleet:

`EV_SIM_FLEET_COUNT=50 EV_SIM_ACCELERATED=true npm run dev fleet`

Best fit

The simulator is ideal for EV charging platforms, CPO engineering teams, integrators, and QA organizations that need dependable, repeatable charger behavior at scale.

OCPP 1.6J • **TypeScript / Node.js** • **WebSocket/JSON** • Works with any ev-server deployment • Docker & CI/CD ready